

JavaScript: Array Methods

1) map vs forEach: Contrast semantics and return values. Show one example where forEach cannot replace map.

1. המטודה map מקבלת פונקציה כארגומנט (קולבק) אותה היא מריצה על כל איבר במערך ומחזירה מערך חדש עם כל התוצאות של פעולת הפונקציה על כל איבר במערך בהתאמה.

המטודה forEach גם מקבלת קולבק ומפעילה אותה על כל איבר במערך. אבל בניגוד ל-map המטודה forEach עצמה לא מחזירה כל ערך.

```
workspace > HomeWork > try5 > JS doogmaot.js > ...
1  const arr = [1, 2, 3, 4, 5]
2  let sum = 0
3  arr.map(x => sum += x)
4  console.log('map(): ', sum);
5
6  let sum2 = 0
7  arr.forEach(x => sum2 += x)
8  console.log('forEach(): ', sum2);
9
10 let sum3 = 0
11 console.log(arr.map(x => sum3 += x));
12
13 console.log(arr.forEach(x => sum3 += x));
14
15 let sum4 = 0
16 arr.map(x => x * 2).forEach(x => sum4 += x)
17 console.log(sum4);
```

בקוד הדוגמא שכתבתי אנחנו רואים כיצד מקבלים את אותה תוצאה כשאנחנו קולבק מופעל בשתי המטודות (שורה 4 ושורה 8) בשתי המטודות שיניתי משתנה ואת תוצאת השינוי הדפסתי.

אבל אם ננסה להדפיס את הערך שהמטודה עצמה מחזירה. נוכל לראות שהמטודה map מחזירה לנו מערך עם חמש תוצאות (שורה 11) שכל אחת מהן היא תוצאת פעולת הקולבק שינוי ערכו של sum3 ע"י תוספת של איבר מהמערך (arr) על כל איבר במערך בהתאמה.

לעומת זאת,

map(): 15	doogmaot.js:4
forEach(): 15	doogmaot.js:8
▶ (5) [1, 3, 6, 10, 15]	doogmaot.js:11
undefined	doogmaot.js:13
30	doogmaot.js:17

הדפסה של הערך שמחזירה forEach נותנת לנו undefined (שורה 13) כי כאמור forEach עצמה לא מחזירה ערך כלשהו.

ובגלל שהמטודות עובדות על מערך (ולא על undefined) לא ניתן לשרשר מטודה ל-map בעוד של-map שמחזירה מערך אפשר כמו שמודגם בשורה 16 (map מחזירה מערך תוצאות שכל אחת גדולה פי שניים מהאיברים במערך המקורי בהתאמה ועל המערך החדש הזה עוברת forEach ומוסיפה כל איבר ממנו ל-sum4 ובשורה 17 הדפסנו את ערכו של sum4 אחרי הפעולות.

השרשור הזה זה משהו שאפשר לעשות עם map אבל לא עם forEach.

JavaScript: Array Methods

2) filter vs reduce (for filtering): Remove odds from [1,2,3,4,5] using both. Which is clearer and why?

// with filter

```
21 // 2.
22
23 const arr = [1,2,3,4,5]
24
25 console.log(arr.filter(num => num % 2 == 0));
```

► (2) [2, 4]

[doogmaot.js:25](#)

// with reduce

```
21 // 2.
22
23 const arr = [1,2,3,4,5]
24
25 // console.log(arr.filter(num => num % 2 == 0));
26
27 console.log(arr.reduce(function(accumulator = [], currentValue){
28     if(currentValue % 2 == 0){
29         accumulator.push(currentValue)
30     }
31     return accumulator
32 },[]))
```

► (2) [2, 4]

[doogmaot.js:27](#)

2. עם שתי המטודות הגענו לאותה תוצאה אבל זה ממש ברור שעם filter הקוד הרבה יותר ברור ופחות מסובך. המטודה filter כפי ששמה מרמז בנויה בדיוק למטרה הזאת. היא עוברת על מערך ומחזירה מערך חדש עם כל התוצאות שעברו את מבחן הקולבק.

לעומת זאת reduce מחזירה את הערך האחרון שיצרה הקולבק, ערך ולא מערך ולכן כדי להציג מערך של תוצאות היינו צריכים לסרבב את הקוד ולגרום לכך שהערך יהיה מערך. קבעתי בקולבק שהפרמטר הראשון יהיה מערך ודחפתי לתוכו את הפרמטר השני שהוא האיבר שעליו עברה המטודה (כמובן רק אם הוא זוגי) והחזרתי את הפרמטר הראשון.

אין סיבה להשתמש עם reduce למטרה זאת.

JavaScript: Array Methods

3) find vs filter: Differences in return type, short-circuiting, and common use cases. When would you prefer findIndex?

3. המטודה find מחזירה את הערך של האלמנט הראשון במערך שעובר את מבחן הקולבק אם אין כזה מחזירה undefined.

המטודה filter מחזירה מערך של כל התוצאות שעברו את מבחן הקולבק ואם אין כאלו לא מחזירה דבר.

בקוד להלן מערך של אובייקטים שבתוכם פרטים של אזרחים. עשיתי לו דיסטרקצ'רינג כדי שיהיה יותר נוח לעבוד איתו

בשורה 42 השתמשתי ב find כדי למצוא ולהדפיס לקונסולה את האזרח הראשון במערך שיש לו id עם ערך 111213. במקרה שאנחנו יודעים שמה שאנחנו מחפשים הוא ייחודי במערך כמו מספר id אפשר להשתמש ב- find כדי למצוא אותו כי בטוח שהערך הראשון שנמצא הוא היחיד. המטודה זיהתה את ה-id כשייך לאובייקט עם הפרטים של האזרח Bob והדפיסה את האובייקט עם הפרטים שלו.

```
38 // 3.
39
40 const citizens = [{ 'id' : 111213 , 'name' : 'Bob', 'gender' : 'male' }, { 'id' :
41 const [citizen001, citizen002, citizen003]=citizens
42 console.log(citizens.find(citizen => citizen.id == 111213));
```

▶ {id: 111213, name: 'Bob', gender: 'male'}

[doogmaot.js:42](#)

בשורות 44 ו-45 השתמשתי ב-find וב-filter בהתאמה כדי למצוא ולהדפיס לקונסולה אזרחים שה-gender שלהם הוא male. המטודה find מצאה את האזרח הראשון שהוא זכר גם במקרה הזה זה האזרח Bob והדפיסה את הפרטים שלו. לעומת זאת המטודה filter מצאה את כל האזרחים הזכרים והחזירה מערך עם האובייקטים שלהם. במקרה הזה יותר הגיוני שנרצה להשתמש ב-filter

```
38 // 3.
39
40 const citizens = [{ 'id' : 111213 , 'name' : 'Bob', 'gender' : 'male' }, { 'id' :
41 const [citizen001, citizen002, citizen003]=citizens
42 // console.log(citizens.find(citizen => citizen.id == 111213));
43
44 console.log(citizens.find(citizen => citizen.gender == 'male'));|
45 console.log(citizens.filter(citizen => citizen.gender == 'male'));
```

▶ {id: 111213, name: 'Bob', gender: 'male'}

[doogmaot.js:44](#)

▼ (3) [{...}, {...}, {...}] ⓘ

[doogmaot.js:45](#)

- ▶ 0: {id: 111213, name: 'Bob', gender: 'male'}
- ▶ 1: {id: 111215, name: 'Patrick', gender: 'male'}
- ▶ 2: {id: 111216, name: 'Sheldon', gender: 'male'}
- length: 3
- ▶ [[Prototype]]: Array(0)

JavaScript: Array Methods

המטודה `findIndex` מוצאת את המקום של האלמנט הראשון במערך שעובר את מבחן הקולבק אם לא נמצא אחד כזה היא מחזירה -1

השתמשי ב- `findIndex` כדי למצוא את המקום של של האובייקט שה- `id` שלו שווה ל- 111213 (שורה 47) אפשר לראות בהדפסה (שורה 48) שהוא 0, ואז החלפתי את האובייקט במקום הזה עם אובייקט הפרטים של האזרחית Sandy. כשהמיקום של משהו ייחודי למערך חשוב לנו אפשר להשתמש ב- `findIndex`.

```
38 // 3.
39
40 const citizens = [{ 'id' : 111213 , 'name' : 'Bob', 'gender' : 'male' }, { 'id' : 111215 , 'n
41 const [citizen001, citizen002, citizen003]=citizens
42 // console.log(citizens.find(citizen => citizen.id == 111213));
43
44 // console.log(citizens.find(citizen => citizen.gender == 'male'));
45 // console.log(citizens.filter(citizen => citizen.gender == 'male'));
46
47 const citizenPlaceInArray = citizens.findIndex(citizen => citizen.id == 111213)
48 console.log(citizenPlaceInArray);
49 citizens[citizenPlaceInArray] = { 'id' : 111214 , 'name' : 'Sandy' , 'gender' : 'female' }
50 console.log(citizens);
```

0

[doogmaot.js:48](#)

▼ (3) [{...}, {...}, {...}] ⓘ

[doogmaot.js:50](#)

- ▶ 0: {id: 111214, name: 'Sandy', gender: 'female'}
- ▶ 1: {id: 111215, name: 'Patrick', gender: 'male'}
- ▶ 2: {id: 111216, name: 'Sheldon', gender: 'male'}
- length: 3
- ▶ [[Prototype]]: Array(0)

JavaScript: Array Methods

4) some vs every: Explain short-circuiting behavior and give a real predicate example for each.

4. המטודה some בודרת אם יש אלמנטים במערך שעונים על התנאי של הקולבק ואם יש לפחות אחד כזה מחזירה true אם אין מחזירה false.

המטודה every עושה את אותו הדבר אבל מחזירה true רק אם כל האלמנטים במערך עונים על התנאי של הקולבק.

בקוד להלן מוצגות התוצאות של הפעלת שתי המטודות על מערך של אובייקטים המכילים שם של תלמיד ואת תשובתו לשאלה האם הבין את ההסבר של המורה. שילבתי את התוצאות כדי ליצור הצהרה של המורה.

```
54 // 4.
55
56 const students = [{ 'name' : 'Bob', 'answer' : 'No' }, { 'name' : 'Sandy', 'answer' : 'Yes' }, { 'name' : 'Sally', 'answer' : 'Yes' }, { 'name' : 'John', 'answer' : 'No' } ]
57 const [student01, student02, student03, student04] = students
58 const somewhat = students.some(student => student.answer == 'Yes')
59 const allWhat = students.every(student => student.answer == 'Yes')
60 console.log(`It will be ${somewhat} to say that some of my students understand my explanation.`);
61 console.log(`It will be ${allWhat} to say that all of them did.`);
```

It will be true to say that some of my students understand my explanation.

[doogmaot.js:60](#)

It will be false to say that all of them did.

[doogmaot.js:61](#)

JavaScript: Array Methods

5) includes vs indexOf: How do they differ in handling NaN? Provide a tiny demo.

```
const xs = [1, NaN, 3];  
// includes?  
// indexOf?
```

5. בניגוד למטודה includes המטודה indexOf מתעלמת מ-NaN לצורך ההדגמה יצרת פונקציה שמתרגמת את התוצאות של indexOf לשפה של אנשים. להלן ההדגמה הקצרה:

```
66 // 5.  
67  
68 const xs = [1, NaN, 3];  
69 console.log(`We have this array: [${xs}]`);  
70 console.log(`We see it is ${xs.includes(NaN)} that this array includes NaN`);  
71 let where = xs.indexOf(NaN)  
72 let where3 = xs.indexOf(3)  
73 translateIndexOf = (answer) => answer == -1 ? 'nowhere to be found' : `located at position ${answer}`  
74 console.log(`And we know the method indexOf() can tell us where it is in the array`);  
75 console.log('For example:');  
76 console.log(`Asking where is 3, gives us the answer: "3 is ${translateIndexOf(where3)}"`);  
77 console.log(`So why, when we're asking where NaN is, the answer is: "NaN is ${translateIndexOf(where)}"?`);
```

We have this array: [1,NaN,3]	doogmaot.js:69
We see it is true that this array includes NaN	doogmaot.js:70
And we know the method indexOf() can tell us where it is in the array	doogmaot.js:74
For example:	doogmaot.js:75
Asking where is 3, gives us the answer: "3 is located at position 2"	doogmaot.js:76
So why, when we're asking where NaN is, the answer is: "NaN is nowhere to be found"?	doogmaot.js:77

6) slice vs splice: Remove the 3rd element from an array in two ways and explain which method mutates.

```
const a = [1,2,3,4];  
// non-mutating remove with slice  
// mutating remove with splice
```

6. הפעם הקוד שכתבתי גם מדגים וגם עונה לשאלה.אני חושב שבמקרה הזה מילים נוספות מיותרות. הקוד ותוצאותיו בעמוד הבא

JavaScript: Array Methods

```
82 // 6.
83
84 const a = [1,2,3,4];
85 console.log(`The original array [${a}] before slice()`);
86 console.log('Using slice() to remove the 3rd element');
87 console.log('The method creates a new array for each slice that was copied:');
88 console.log(a.slice(0,2), a.slice(3));
89 console.log('A new array created by me containing what was copied:');
90
91 const aShortendBySlice = [...a.slice(0,2), ...a.slice(3)]
92
93 console.log(aShortendBySlice);
94 console.log(`The original array [${a}] is unchanged`);
95
96 console.log('-----');
97
98 console.log(`The original array [${a}] before splice()`);
99 console.log('Using splice() to remove the 3rd element');
100 console.log('The method creates a new array for each slice that was cut out:');
101
102 console.log(a.splice(2,1));
103
104 console.log(`The original array [${a}] is changed`);
```

The original array [1,2,3,4] before slice()

[doogmaot.js:85](#)

Using slice() to remove the 3rd element

[doogmaot.js:86](#)

The method creates a new array for each slice that was copied:

[doogmaot.js:87](#)

▸ (2) [1, 2] ▸ [4]

[doogmaot.js:88](#)

A new array created by me containing what was copied:

[doogmaot.js:89](#)

▸ (3) [1, 2, 4]

[doogmaot.js:93](#)

The original array [1,2,3,4] is unchanged

[doogmaot.js:94](#)

[doogmaot.js:96](#)

The original array [1,2,3,4] before splice()

[doogmaot.js:98](#)

Using splice() to remove the 3rd element

[doogmaot.js:99](#)

The method creates a new array for each slice that was cut out:

[doogmaot.js:100](#)

▸ [3]

[doogmaot.js:102](#)

The original array [1,2,4] is changed

[doogmaot.js:104](#)

JavaScript: Array Methods

7) sort pitfalls: Why does `['10','2','1'].sort()` produce a surprising order? Provide two correct numeric comparators (`asc/desc`).

```
// asc  
// desc
```

7. הסיבה לסדר המפתיע כשמשתמשים ב- `sort` על מספרים היא ש-`sort` מתייחס למספרים כאילו הם `string` ואז מבחינתה כל מה שמתחיל ב-1 קטן ממה שמתחיל ב-2 וכך היא משווה בין המספר הנוכחי במערך 10 למספר הבא 2, קובעת ש-2 גדול יותר ושולחת אותו לסוף המערך ואז משווה בין המספר הנוכחי 10 למספר הבא 1 קובעת ש-10 גדול יותר ושולחת אותו לסוף וכך מתקבלת התוצאה בשורה 114.

המטודה `sort` מקבלת קולבק ואפשר להשתמש בה כדי לקסל את התוצאה הרצויה. בשורה 116 בגלל הקולבק `sort` משווה בין המספרים ע"י חיסור המספר הבא מהמספר המוכחי. אם התוצאה הייתה שלילית היא הייתה שולחת את המספר הבא לסוף. אם התוצאה חיובית כמו במקרה שלנו היא שולחת את המספר הנוכחי (10) הגדול יותר לסוף המערך. והתוצאה בסופו של דבר מספרים מהקטן לגדול (שורה 116).

אם היינו רוצים שהמספרים יהיו מהגדול לקטן פשוט נהפוך בקולבק למספר הבא פחות המספר הנוכחי. (שורה 118)

אפשרות נוספת היא לעשות את מה שעשינו בשורה 116 ואז להפעיל את המטודה `reverse` שהופכת את הסדר במערך (שורה 120).

```
109 // 7.  
110  
111 const unsortedArr = [10, 2, 1]  
112 console.log(unsortedArr);  
113  
114 console.log(unsortedArr.sort());  
115  
116 console.log(unsortedArr.sort((num1, num2) => num1 - num2));  
117  
118 console.log(unsortedArr.sort((num1, num2) => num2 - num1));  
119  
120 console.log(unsortedArr.sort((num1, num2) => num1 - num2).reverse());
```

‣ (3) [10, 2, 1]	doogmaot.js:112
‣ (3) [1, 10, 2]	doogmaot.js:114
‣ (3) [1, 2, 10]	doogmaot.js:116
‣ (3) [10, 2, 1]	doogmaot.js:118
‣ (3) [10, 2, 1]	doogmaot.js:120

JavaScript: Array Methods

8) localeCompare for strings: Sort ['é','e','ê'] correctly for French and then reverse it.

```
// ascending (fr)
// descending (fr)
```

.8

```
125 // 8.
126
127 const theFrenchArr = ['é','e','ê']
128
129 console.log('The original array:');
130 console.log(theFrenchArr);
131
132 console.log('ascending (fr):');
133 console.log(theFrenchArr.sort((letter1, letter2) => letter1.localeCompare(letter2, "fr")));
134
135 console.log('descending (fr):');
136 console.log(theFrenchArr.sort((letter1, letter2) => letter2.localeCompare(letter1, "fr")));
```

The original array:

[doogmaot.js:129](#)

▶ (3) ['é', 'e', 'ê']

[doogmaot.js:130](#)

ascending (fr):

[doogmaot.js:133](#)

▶ (3) ['e', 'é', 'ê']

[doogmaot.js:134](#)

descending (fr):

[doogmaot.js:136](#)

▶ (3) ['ê', 'é', 'e']

[doogmaot.js:137](#)

JavaScript: Array Methods

9) push/pop vs shift/unshift: Compare typical uses and performance implications in large arrays.

9. המטודה push מכניסה אלמנטים לסוף המערך. המטודה pop מסירה את האלמנט האחרון מהמערך. המטודה unshift מכניסה אלמנטים לתחילת המערך. המטודה shift מסירה את האלמנט הראשון במערך. כל ארבע המטודות משנות את המערך המקורי. המטודות pop ו-push יעילות יותר במערכים גדולים כי גודל המערך אינו משנה את זמן הביצוע. המטודות shift ו-unshift מזיזות את כל האלמנטים במערך מקום אחד אחורה או קדימה וזה הופך אותן לפחות יעילות במערכים גדולים. ככל שהמערך יותר גדול צריך להזיז יותר אלמנטים וזמן הביצוע יותר ארוך.

10) Chaining: Transform an orders array into the total price of in-stock items over \$50 using map/filter/reduce in a single chain.

// orders: [{price, inStock}, ...]

.10

```
136 const orders = [  
137   { price: 100, inStock: "Yes" },  
138   { price: 150, inStock: "No" },  
139   { price: 25, inStock: "No" },  
140   { price: 50, inStock: "Yes" },  
141   { price: 30, inStock: "Yes" },  
142   { price: 125, inStock: "Yes" },  
143   { price: 40, inStock: "Yes" },  
144   { price: 60, inStock: "Yes" }  
145 ];  
146 console.log(orders);  
147  
148 console.log(  
149   orders  
150     .map((item) => (item.inStock == "Yes" ? item.price : item.inStock))  
151     .filter((item) => item > 50)  
152     .reduce((total, currentItem) => {  
153       return total + currentItem  
154     })  
155 );
```

▼ (8) [{...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}] i

[doogmaot.js:147](#)

- ▶ 0: {price: 100, inStock: 'Yes'}
- ▶ 1: {price: 150, inStock: 'No'}
- ▶ 2: {price: 25, inStock: 'No'}
- ▶ 3: {price: 50, inStock: 'Yes'}
- ▶ 4: {price: 30, inStock: 'Yes'}
- ▶ 5: {price: 125, inStock: 'Yes'}
- ▶ 6: {price: 40, inStock: 'Yes'}
- ▶ 7: {price: 60, inStock: 'Yes'}
- length: 8
- ▶ [[Prototype]]: Array(0)

JavaScript: Array Methods

11) Stability of sort: Is Array.prototype.sort stable in modern JS? Show how to keep original order for items with equal keys when building a comparator.

11. ב-JS מודרני sort אמורה להיות יציבה. כלומר אם מסדרים איברים במערך לפי ערכם ויש כמה אלמנטים עם ערכים זהים הסידור ביניהם יהיה לפי המיקום שלהם במערך. אם רוצים להיות בטוחים אפשר להשתמש בקוד להלן.

בשאלה נעשה שימוש ב-Array.prototype זה משמש כדי להוסיף מטודה למערך. יצרתי מטודה sortIt כי לא רציתי שיהיה בלבול עם המטודה הקיימת sort והוספתי אותה למערך.

הקוד יוצר בעזרת map (שבפרמטר השני שלה יודעת לקבל את המיקום של האלמנט במערך) מערך עם key נוסף oldIndex שהערך שלו זהה למיקום של האלמנט במערך המקורי. בהמשך הקוד המערך החדש מסודר עם sort בדומה למה שעשיתי בתשובה לשאלה 7 אם המחירים אינם זהים אז לפי המחירים ואם כן אז לפי המיקומים. בסוף שוב משתמשים ב-map כדי למחוק מהמערך המסודר את oldIndex ומדפיסים לקונסולה.

הפריטים עם המחיר הזהה (504 ו-660) מסודרים לפי מיקומם במערך המקורי.

```
1  const itemAndPrice = [
2    {item: 504, price: 50},
3    {item: 512, price: 100},
4    {item: 660, price: 50},
5    {item: 666, price: 10},
6  ]
7  Array.prototype.sortIt = function() {
8    const itemAndPriceAndOldIndex = this.map((item, index) => ({item, oldIndex : index}))
9    itemAndPriceAndOldIndex.sort((elm1, elm2) => {
10      if(elm1.item.price !== elm2.item.price){
11        return elm1.item.price - elm2.item.price
12      }
13      return elm1.oldIndex - elm2.oldIndex
14    })
15    const sortedItemAndPrice = itemAndPriceAndOldIndex.map(a => a.item)
16    console.log(sortedItemAndPrice);
17  }
18  itemAndPrice.sortIt()
```

▼ (4) [{...}, {...}, {...}, {...}] ⓘ

- ▶ 0: {item: 666, price: 10}
- ▶ 1: {item: 504, price: 50}
- ▶ 2: {item: 660, price: 50}
- ▶ 3: {item: 512, price: 100}

length: 4

▶ [[Prototype]]: Array(0)

[doogmaot.js:16](#)

JavaScript: Array Methods

12) Performance thought exercise: You need to append many small arrays into one big array in a loop. Compare: `big = big.concat(small)` vs `big.push( .small)` vs `prealloc + index writes`. Pros/cons?

12. אם נשתמש ב-`concat` כדי ליצור מערך גדול מהרבה מערכים קטנים בלולאה זה יכביד על הזכרון בגלל שבכל איטרציה `concat` תיצור מערך חדש ותעתיק את האלמנטים בו למערך הגדול. אם נעשה את זה עם `push` בשילוב עם (...) שרק מוסיפה אלמנט לסוף מערך הגדול המקורי בכל איטרציה זה יהיה יותר יעיל. לגבי שמירת אינדקסים פנויים במערך המקורי והוספה לשם זה לא ישים עם השיטות האלה כי `concat` בכל מקרה יוצרת מערך חדש ואז אנחנו עושים השמה *by reference* למערך הגדול (ודורסים את המקומות הריקים) ו-`push` היא מוסיפה לסוף המערך המקורי כלומר אחרי כל מקום ריק שנשאר בו.

23) Safety with nested data: Given an array of users where some have `addresses: []` or missing, count total addresses using `flatMap` safely without throwing.

```
// users: [{addresses?: [{...}]}, ...]
```

```
53  const users = [  
54    {'name': 'Bob' , 'address': [{ 'St.': '1st', 'city': 'city0' } ]},  
55    {'name': 'Patrick' , 'address': []},  
56    {'name': 'Sandy' , 'address': [{ 'St.': '1st', 'city': 'city0' } ]},  
57    {'name': 'Sheldon'},  
58    {'name': 'Hugine' , 'address': [{ 'St.': '1st', 'city': 'city0' } ]},  
59  ]  
60  
61  const howManyAddresses = users.flatMap(x => Object.entries(x))  
62    .filter(x => x[0] === 'address')  
63    .map(x => x.find(x => x == ''))  
64    .filter(x => x === undefined).length  
65  
66  console.log('Total addresses: ', howManyAddresses);
```

Total addresses: 3

[doogmaot.js:66](#)

23. אחרי שעשיתי `flatMap` השתמשתי ב-`Object.entries` כדי ליצור מערך (כרגע יש לי אובייקטים) שאוכל להפעיל עליו את `filter` ולסנן רק את האובייקטים של `address` אז עשיתי `map` כדי להפעיל על כל אלמנט במערך שקיבלתי את `find` ולמצוא את הערכים הריקים זה נתן לי מערך עם כל הערכים הריקים של `address` (יש אחד כזה) ושאר הערכים היו `undefined` כי `map` הפעילה את `find` על כל האלמנטים ומה שלא עונה לקולבק של `find` חוזר כ-`undefined` בסוף הפעלתי שוב `filter` וסיננתי את מה שריק (מה שלא `undefined`) והדפסתי את אורך המערך ששווה לכל הכתובות שהן לא []